



RTL Design Sherpa

KESTREL-RV32I Micro-Architecture Specification 0.1

October 7, 2026

Table of Contents

1 Kestrel Mas Index.....	11
2 Document Information.....	12
2.1 KESTREL-RV32I Micro-Architecture Specification.....	12
2.2 Document Purpose.....	12
2.3 References.....	12
2.4 Terminology.....	13
2.5 Revision History.....	13
3 Design Philosophy and Machine Organization.....	15
3.1 Design Philosophy.....	15
3.2 The Machine in One Picture.....	16
3.2.1 Figure 1.1: kestrel datapath block diagram.....	16
3.3 Module Inventory.....	16
3.4 State Inventory.....	17
3.5 Key Interfaces at a Glance.....	18
4 kestrel_core (Top).....	20
4.1 Purpose.....	20
4.2 Interface.....	20
4.3 Internal Structure.....	20
4.3.1 Fetch.....	20
4.3.2 Source Muxes.....	20
4.3.3 Writeback Priority Mux.....	20
4.3.4 Next-PC Mux.....	21

4.3.5 Branch Comparator (the Ruling).....	21
4.3.6 Load/Store Rotation Network.....	22
4.3.7 Halt and PC Registers.....	22
5 kestrel_decode.....	24
5.1 Purpose.....	24
5.2 Keying and Defaults.....	24
5.3 The Decode Table.....	25
5.3.1 Arithmetic and upper-immediate rows.....	25
5.3.2 Loads, stores, branches, jumps.....	26
5.3.3 System, fence, and the default.....	28
5.4 What the Table Teaches.....	29
6 kestrel_imm_gen.....	30
6.1 Purpose.....	30
6.2 Interface.....	30
6.3 Internal Structure.....	30
7 kestrel_alu.....	32
7.1 Purpose.....	32
7.2 Interface.....	32
7.3 Operations.....	32
7.4 The Flags Ruling.....	33
8 kestrel_regfile.....	34
8.1 Purpose.....	34
8.2 Interface.....	34
8.3 Internal Structure and Rules.....	34

9	kestrel_mem_loader.....	36
9.1	Purpose.....	36
9.1.1	Figure 2.1: Loader composition.....	36
9.2	Memory Arrays.....	36
9.3	CTRL / Run Register and Reset Sequencing.....	37
9.4	Write Side: Leaf Slave plus Two Holding Bits.....	37
9.5	Read Side: Combinational Beat through the AR Skid.....	37
9.6	Address Map, as Implemented.....	38
9.7	Debug Taps.....	38
10	Package Types and Top-Level Wiring.....	39
10.1	kestrel_pkg.....	39
10.1.1	ALU operation enum (alu_op_e, 4 bits).....	39
10.1.2	Immediate-format enum (imm_sel_e, 3 bits).....	39
10.1.3	Halt-cause encodings (localparam logic [3:0]).....	39
10.2	Top-Level Wiring (kestrel_core).....	40
10.3	Reset Discipline.....	41
11	The Single Cycle, Step by Step.....	42
11.1	The Clock in One Sentence.....	42
11.2	1. Fetch and Decode.....	42
11.3	2. Operands and Immediates.....	42
11.4	3. Execute.....	42
11.5	4. Memory Access.....	42
11.6	5. Writeback.....	43
11.7	6. Next-PC Selection and the Edge.....	43

11.8 The Basic Cycle on the Pins.....	43
11.8.1 Waveform 4.1: Basic single-cycle execution — an aligned load followed by an ALU op.....	43
11.9 Branch and Jump Resolution.....	44
12 Cross-Word Misaligned Retry.....	45
12.1 When the Retry Fires.....	45
12.2 Beat-by-Beat Behavior.....	45
12.3 The Waveform.....	46
12.3.1 Waveform 4.2: Cross-word store retry — SH at byte offset 3.....	46
12.4 Worked Example: a Halfword Across the Boundary.....	46
12.5 Why Hardware, Why Two Cycles.....	47
13 Halt, IALIGN, and the System Layer.....	48
13.1 The Halt Holding Register.....	48
13.2 The IALIGN Halt (Cause 3).....	48
13.3 The System Layer: Stubs, NOPs, and Halts.....	49
13.3.1 FENCE and FENCE.I retire as NOPs.....	49
13.3.2 MRET falls through.....	49
13.3.3 The CSR class retires as a bounded stub.....	49
13.4 Halt Timing Summary.....	50
14 RVFI Aggregation.....	51
14.1 Retirement as the Observation Channel.....	51
14.2 The Field Table.....	51
14.3 The Three Gating Rules.....	52
14.4 Memory-Field Packing.....	52

14.5 Why This Pays.....	52
15 Verification Hooks and Observability.....	54
15.1 The Three Testplan-Tracked Hooks.....	54
15.2 Supporting Hooks.....	55
15.3 The Consumer Stack.....	56
15.4 Cross-Reference to the Testplan Set.....	56
16 Parameters, Defines, and Filelists.....	58
16.1 Parameters.....	58
16.1.1 kestrel_core.....	58
16.1.2 kestrel_mem_loader.....	58
16.2 Defines and Includes.....	59
16.3 Filelists.....	59
16.4 Regenerating This Document.....	60
17 KESTREL-RV32I Hardware Architecture Specification Index.....	61
17.1 Document Organization.....	61
17.1.1 Front Matter.....	61
17.1.2 Chapter 1: Introduction.....	61
17.1.3 Chapter 2: System Overview.....	61
17.1.4 Chapter 3: Architecture.....	61
17.1.5 Chapter 4: Interfaces.....	61
17.1.6 Chapter 5: Performance.....	61
17.1.7 Chapter 6: Integration.....	61
17.2 Related Documentation.....	62
18 Simplified RV32I: kestrel.....	63

18.1 Overview.....	63
18.2 Document Structure.....	63
18.2.1 Chapter 1: Introduction.....	63
18.2.2 Chapter 2: The RV32I ISA Quick Reference.....	64
18.2.3 Chapter 3: The kestrel Datapath.....	64
18.2.4 Chapter 4: The Control Truth Table.....	64
18.2.5 Chapter 5: The Memory Contract.....	64
18.2.6 Chapter 6: Verification.....	64
18.2.7 Chapter 7: What kestrel Costs.....	65
18.2.8 Version History.....	65
19 kestrel-rv32i Testplans.....	66
19.1 Inventory.....	66
19.2 What each plan covers.....	67
19.3 JUnit naming convention (observed).....	67
19.4 Rollup.....	68
19.5 Running the tests.....	68
19.6 Functional coverage (CORE-18).....	69

List of Figures

Figure 1.1: kestrel datapath block diagram.....	16
Figure 2.1: Loader composition.....	36

List of Tables

Table 1: Reference Documents.....	12
Table 2: Terminology and Definitions.....	13
Table 3: Document Revision History.....	13
Table 4: Module inventory.....	16
Table 5: The complete core state inventory, register file aside.....	18
Table 6: Internal interface summary (full wiring in Chapter 3).....	18
Table 7: Writeback priority mux.....	20
Table 8: The thirteen decode outputs.....	24
Table 9: Decode rows — OP, OP-IMM, LUI, AUIPC.....	25
Table 10: Decode rows — LOAD, STORE, BRANCH, JAL, JALR.....	26
Table 11: Decode rows — SYSTEM, MISC-MEM, and the illegal default.....	28
Table 12: <code>kestrel_imm_gen</code> interface.....	30
Table 13: Immediate assembly per format.....	30
Table 14: <code>kestrel_alu</code> interface.....	32
Table 15: ALU operations (<code>shamt = b[4:0]</code>).....	32
Table 16: <code>kestrel_regfile</code> interface.....	34
Table 17: Loader map implementation facts.....	38
Table 18: <code>alu_op_e</code> enumerators.....	39
Table 19: <code>imm_sel_e</code> enumerators.....	39
Table 20: <code>HALT_*</code> encodings — the single source of truth shared by decode and core.....	39
Table 21: <code>kestrel_core</code> internal wiring map.....	40
Table 22: Control-transfer resolution.....	44
Table 23: The two beats of a cross-word access (<code>o = addr[1:0]</code>).....	45
Table 24: Halt timing at the boundary.....	50
Table 25: RVFI signals and their kestrel aggregation.....	51
Table 26: The three testplan-tracked hooks.....	54
Table 27: Supporting loader hooks.....	55
Table 28: Verification consumer stack.....	56
Table 29: <code>kestrel_core</code> parameters.....	58
Table 30: <code>kestrel_mem_loader</code> parameters.....	58
Table 31: Shared defines.....	59
Table 32: Filelists (<code>KESTREL_ROOT = projects/components/riscv-ip/kestrel-rv32i</code> ; <code>REPO_ROOT = RTLDesignSherpa checkout root</code>).....	59
Table 33: Version history.....	65

List of Waveforms

Waveform 4.1: Basic single-cycle execution — an aligned load followed by an ALU op.....	43
Waveform 4.2: Cross-word store retry — SH at byte offset 3.....	46

1 Kestrel Mas Index

Generated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

2 Document Information

2.1 KESTREL-RV32I Micro-Architecture Specification

Document Number: KESTREL-MAS-001 **Version:** 0.1 **Status:** Draft **Classification:** Open Source - MIT License

2.2 Document Purpose

This Micro-Architecture Specification (MAS) describes how the kestrel-rv32i core is built: one section per functional block covering its purpose, interface, internal structure, and key logic; the internal interfaces between blocks; the cycle-level behavior including the misaligned retry and halt timing; the verification hooks the design exposes on purpose; and the configuration surface. It is written for implementers and maintainers of the RTL, and for reviewers who need to check the design against its specification.

The external contract — ports as an integrator wires them, memory and halt behavior, loader programming — is the companion Hardware Architecture Specification (HAS). This document assumes it.

Target Audience: - RTL maintainers extending or fixing the core - Design reviewers checking implementation against intent - Verification engineers wiring checkers to the internal observability points - Students reading the RTL top to bottom

Companion Documents: - KESTREL-RV32I Hardware Architecture Specification (HAS) - External architecture and integration contract - Simplified RV32I: kestrel - Tutorial-study book covering the same ground as pedagogy

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	KESTREL HAS v0.1	Hardware Architecture Specification (companion)
[REF-2]	RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (projects/components/riscv-ip/references/riscv-	ISA authority; cited by chapter/section as “unpriv \$N”

ID	Document	Description
	spec.pdf)	
[REF-3]	riscv-formal rvfi.md (vendor/riscv-formal/docs/rvfi.md)	RVFI retire-channel field semantics
[REF-4]	KESTREL testplans (dv/testplans/*.yaml)	V&V scenarios and coverage cross-reference
[REF-5]	falcon-suite RTL style brief	Datapath-as-truth-tables / FSMs-only-where-unavoidable discipline

2.4 Terminology

Terminology and Definitions

Term	Definition
Beat	One memory access cycle; a cross-word access uses two beats
Control bundle	The thirteen-field decode output steering the datapath each cycle
FUB	Functional Unit Block - leaf RTL module
Halt holding register	halt_q — one bit that latches any halt cause and holds it
Retry bit	ls_retry — one bit of control state sequencing the cross-word retry
Rotator	The byte-lane shift network implementing rotated strobes/data for L/S
Trap beat	The single RVFI beat of a halting instruction (rvfi_trap=1)

2.5 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-07	seang	Initial MAS release; documents the as-built RTL

Version	Date	Author	Description
			(Tasks 1-11) including the Task-9 IALIGN halt and the Task-11 AXIL board loader

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

3 Design Philosophy and Machine Organization

3.1 Design Philosophy

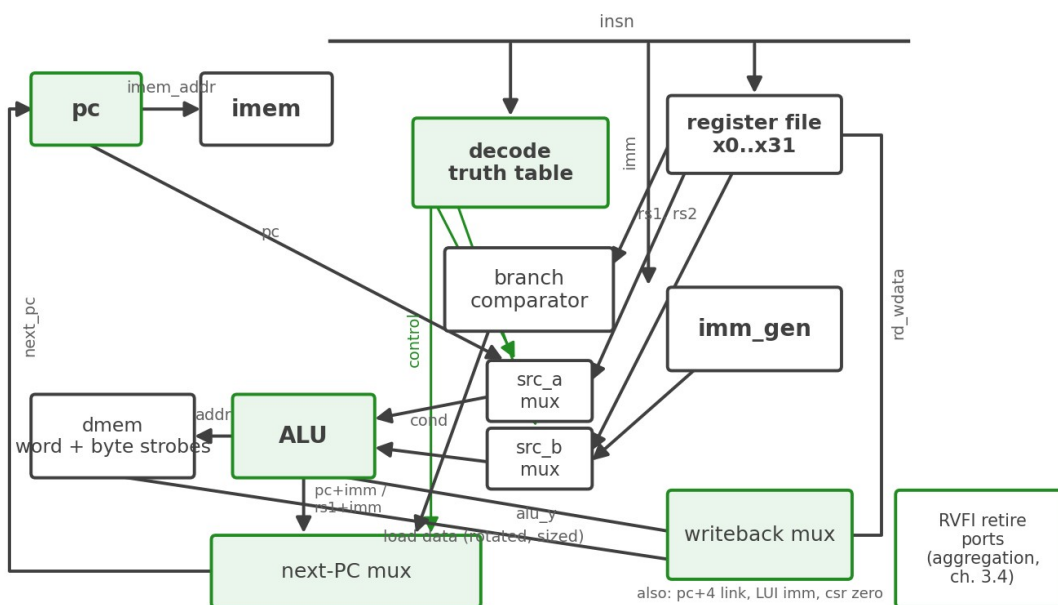
Three commitments shape the microarchitecture, and every block chapter shows them landing:

1. **The datapath is FSM-free.** Classic processor control walks fetch/decode/execute/memory/writeback as states; in kestrel those phases are spatial, not temporal. Fetch is a wire off the PC register; decode is a truth table; execute is an ALU; memory access is a combinational port; writeback is a mux feeding a clocked register file. There is nothing to sequence, so there is no FSM — and no microcode, no control ROM, no phase counter. The two places a naive design would reach for an FSM — halting and the misaligned retry — are single holding flops with plain set/clear conditions.
2. **Control is an explicit truth table.** Decode is one unique casez keyed on {opcode, funct3, funct7} producing a thirteen-field control bundle. The same word always produces the same bundle; the table is small enough to read top to bottom and is transcribed in the kestrel_decode chapter.
3. **RVFI is the observation channel.** The retire port is not debug instrumentation bolted on afterward; it is the defined observation channel, present from the first vertical slice, and its aggregation rules are stated next to the assignments in the RTL and again in Chapter 4.

The discipline the suite's style brief imposes — datapath logic is truth tables and muxes; FSMs only where control is unavoidable — is what makes this core reviewable in one sitting and provable at shallow model-checking depth (riscv-formal runs at depth 6 and covers reset plus a two-cycle memory access with headroom).

3.2 The Machine in One Picture

3.2.1 Figure 1.1: kestrel datapath block diagram



kestrel datapath block diagram

The diagram is drawn to match the RTL: the blocks are the module instances in `kestrel_core.sv`, and every named wire is a real signal in the source. State elements are the PC register, the register file, and three small holding registers described below; everything else is combinational.

3.3 Module Inventory

Module inventory

Module	File	Role	State
kestrel_pkg	rtl/includes/kestrel_pkg.sv	Shared enums (<i>alu_op_e</i> , <i>imm_sel_e</i>) and the halt-cause encodings	none
kestrel_decode	rtl/fub/kestrel_decode.sv	Combinational truth table from { <i>opcode</i> , <i>funct3</i> ,	none

Module	File	Role	State
		funct7} to the control bundle	
kestrel_imm_gen	rtl/fub/kestrel_imm_gen.sv	Builds the 32-bit immediate for the five formats	none
kestrel_alu	rtl/fub/kestrel_alu.sv	Ten operations: ADD SUB AND OR XOR SLL SRL SRA SLT SLTU	none
kestrel_regfile	rtl/fub/kestrel_regfile.sv	32 x 32-bit registers, two combinational read ports, one synchronous write, x0 discarded	32 words
kestrel_core	rtl/top/kestrel_core.sv	Wires the leaves together: source muxes, branch comparator, next-PC mux, writeback mux, L/S rotation, halt, RVFI	5 elements below
kestrel_mem_loader	rtl/fub/kestrel_mem_loader.sv	Optional board glue: memories + AXIL slave + run control (see its chapter)	run bit, holding regs

3.4 State Inventory

Everything kestrel remembers, outside the register file and (when integrated) the memories:

The complete core state inventory, register file aside

Element	Width	Purpose	Next-state equation
pc	32	Program counter; resets to RESET_ADDR	$pc \leq halt ? pc : (ls_first ? pc : next_pc)$
retire_count	64	RVFI retirement order: 0, 1, 2, ...	+1 when rvfi_valid
halt_q	1	Halt holding register; latches any halt cause	$halt_q \leq halt_q \vee halt_now$
ls_retry	1	Cross-word L/S retry-cycle flag	$ls_retry \leq ls_first$
ls_rdata_lo	32	First-word read data captured for the retry-cycle merge	captures ls_rdata_shifted when ls_first

Five state elements with one-line next-state equations are reviewable in minutes. The retry bit and the halt bit are control state only; ls_rdata_lo is the only place data is captured mid-instruction — a pipeline register in the most minimal sense.

3.5 Key Interfaces at a Glance

Internal interface summary (full wiring in Chapter 3)

Interface	Width	Producer → Consumer
Control bundle	13 fields	kestrel_decode → muxes, regfile, L/S datapath
imm	32	kestrel_imm_gen → source-B mux, writeback (LUI)
alu_y	32	kestrel_alu → dmem address, writeback
Branch flags	eq/lt/ltn	dedicated comparator → branch-cond

Interface	Width	Producer → Consumer
		mux → next-PC mux
next_pc	32	next-PC mux → PC register and rvfi_pc_wdata
rd_wdata	32	writeback mux → register file
RVFI bundle	17 signals	aggregation assigns → top-level ports

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 kestrel_core (Top)

4.1 Purpose

`kestrel_core` wires the four datapath leaves together and owns everything that is not a leaf: source muxes, the dedicated branch comparator, the next-PC mux, the writeback priority mux, the load/store rotation network with its retry state, the halt holding register, and the RVFI aggregation assigns. It is the only module in the core with architectural state besides the register file.

4.2 Interface

Ports are specified exhaustively in the HAS (Chapter 4 there). Internally the top exposes no hierarchy below the four leaf instances (`u_decode`, `u_imm_gen`, `u_regfile`, `u_alu`) plus its own always-blocks; the loader, when used, sits outside this module.

4.3 Internal Structure

4.3.1 Fetch

`imem_addr = pc` and `insn = imem_rdata` — fetch is a wire. The instruction is expected combinationally (the memory contract, HAS Ch. 4); `opcode = insn[6:0]` and `funct3 = insn[14:12]` fan out to decode and the L/S size/sign logic.

4.3.2 Source Muxes

`alu_src_a = alu_src_a_pc ? pc : rs1_data` selects PC (AUIPC, branch targets) or `rs1`.
`alu_src_b = alu_src_b_imm ? imm : rs2_data` selects the immediate or `rs2`. Register-file read addresses come straight from the instruction: `rs1 = insn[19:15]`, `rs2 = insn[24:20]`, `rd = insn[11:7]`.

4.3.3 Writeback Priority Mux

`rd_wdata` is selected by a five-row priority mux:

Writeback priority mux

Priority	Condition	Value
1	<code>csr_stub</code>	32'd0 — the CSR stub's zero writeback
2	<code>opcode == OPCODE_LUI</code>	<code>imm</code> — LUI never passes through the ALU

Priority	Condition	Value
3	ls_load	ls_load_data — assembled, size/sign-corrected read data
4	jump \\ \\ jalr	pc + 4 — the link value
5	default	alu_y

The write enable is $rd_wen_eff = rd_wen \ \& \ \sim ls_first \ \& \ \sim halt$: decode's rd_wen qualified by the cross-word retry's first cycle (a partial word must not commit early) and by $halt$ (the cause-3 $halt$ retires JAL/JALR encodings whose decode rd_wen is set, so the gate is $halt$ itself, not decode). The register file independently discards writes to $x0$.

4.3.4 Next-PC Mux

Four rows, in priority order: JAL/JALR ($jump \ || \ jalr \rightarrow jalr \ ? \ (rs1_data + imm) \ \& \ 32'hFFFF_FFFE : pc + imm$; taken branch ($branch_taken \rightarrow pc + imm$; default $\rightarrow pc + 4$). JALR_ALIGN_MASK = $32'hFFFF_FFFE$ clears target bit 0 per unpriv §2.1.5.1. In a single-cycle machine the branch decision and the $pc + imm$ target settle in parallel, so taken and not-taken branches cost the same cycle.

4.3.5 Branch Comparator (the Ruling)

The ALU exports $eq/lt/ltu$ flags, and the core's ALU instance **deliberately leaves them unconnected** ($.eq \ ()$, $.lt \ ()$, $.ltu \ ()$). The reason is a reviewed design ruling: for branches the ALU's left input is the PC (it is simultaneously computing $pc + imm$ for the next-PC mux), so its internal flags would compare pc against imm , not $rs1$ against $rs2$ — silently wrong branch decisions. Branch conditions therefore come from a dedicated comparator on the register data:

- $cmp_eq = (rs1_data == rs2_data)$
- $cmp_lt = (\$signed(rs1_data) < \$signed(rs2_data))$
- $cmp_ltu = (rs1_data < rs2_data)$
- $branch_cond$ selected from cmp_* by $funct3$ in a unique case (BEQ $\rightarrow eq$, BNE $\rightarrow \sim eq$, BLT $\rightarrow lt$, BGE $\rightarrow \sim lt$, BLTU $\rightarrow ltu$, BGEU $\rightarrow \sim ltu$, default $\rightarrow 0$)
- $branch_taken = branch \ \& \ branch_cond$

In a single-cycle machine this parallel evaluation is free: both the ALU result and the comparator decision are ready when the next PC is chosen.

4.3.6 Load/Store Rotation Network

The L/S datapath implements the hardware-misaligned policy (unpriv §2.1.6 permits hardware handling; the plan records it as a decision). For an access at byte offset $o = \text{alu_y}[1:0]$ with size s bytes:

- **Size decode:** dmem_size (00 byte, 01 half, 10 word) $\rightarrow$ ls_size_bytes (1/2/4), ls_size_mask (0001/0011/1111), ls_data_mask (0xFF/0xFFFF/0xFFFFFFFF).
- **Split arithmetic:** $\text{ls_tail_size} = 4 - o$; $\text{ls_head_size} = s - \text{ls_tail_size}$; $\text{ls_crossing} = \text{ls_active} \& (o + s > 4)$; $\text{ls_first} = \text{ls_crossing} \& \sim \text{ls_retry}$; $\text{ls_second} = \text{ls_retry}$; $\text{ls_single} = \text{ls_active} \& \sim \text{ls_crossing}$.
- **Address:** $\text{dmem_addr} = \text{ls_second} ? \{\text{alu_y}[31:2] + 1, 2'b00\} : \{\text{alu_y}[31:2], 2'b00\}$ — beat 2 issues at the next word.
- **Store rotation (the store rotator):** beat 1 writes $\text{rs2_data} \ll (8*o)$ with strobe $\text{ls_size_mask} \ll o$; beat 2 writes $\text{rs2_data} \gg (8*\text{ls_tail_size})$ with strobe ls_head_strobe (a head-size decode: 0/1/2/3/4 bytes $\rightarrow$ 0000/0001/0011/0111/1111).
- **Load rotation (the load rotator):** beat 1 captures $\text{ls_rdata_lo} \leq \text{dmem_rdata} \gg (8*o)$; beat 2 assembles $(\text{dmem_rdata} \& \text{ls_head_mask}) \ll (8*\text{ls_tail_size}) \mid (\text{ls_rdata_lo} \& \text{ls_tail_mask})$. Single-beat loads use $(\text{dmem_rdata} \gg (8*o)) \& \text{ls_data_mask}$.
- **Size/sign select:** ls_load_data from ls_raw_rdata per funct3: LB sign-extends from bit 7, LBU zero-extends, LH/LHU from bit 15, LW passes through.

$\text{dmem_wstrb} = \text{ls_store} ? (\text{ls_second} ? \text{ls_head_strobe} : \text{ls_size_mask} \ll o) : 4'b0000$. Bytes that would rotate past bit 31 on beat 1 are dropped by construction — beat 2 carries them. Full cycle-level behavior and the waveform are in Chapter 4.

4.3.7 Halt and PC Registers

The halt path (halt_q , misalign_target , halt_cause_eff , halt_now) and the PC register ($\text{pc} \leq \text{halt} ? \text{pc} : (\text{ls_first} ? \text{pc} : \text{next_pc})$) with the retry/merge/counter flops complete the state. Their behavior is specified in Chapter 4.

Last Updated: 2026-10-07

5 kestrel_decode

5.1 Purpose

`kestrel_decode` is a pure function from the 32-bit instruction word to the control bundle. It holds no state and no clock; the same word always produces the same bundle. The bundle's thirteen fields:

The thirteen decode outputs

Field	Type	Meaning when set / value
<code>alu_op</code>	<code>alu_op_e</code>	ALU operation: ADD SUB AND OR XOR SLL SRL SRA SLT SLTU
<code>imm_sel</code>	<code>imm_sel_e</code>	Immediate format to build: I S B U J
<code>alu_src_a_pc</code>	bit	ALU left input is the PC (else rs1)
<code>alu_src_b_imm</code>	bit	ALU right input is the immediate (else rs2)
<code>rd_wen</code>	bit	The instruction writes rd this cycle
<code>dmem_req</code>	bit	A data-memory access is in flight
<code>dmem_we</code>	bit	The access is a store (else a load)
<code>dmem_size</code>	2 bits	Access size: 00 byte, 01 halfword, 10 word
<code>branch</code>	bit	Instruction is a conditional branch
<code>jump</code>	bit	Instruction is JAL
<code>jalr</code>	bit	Instruction is JALR
<code>halt_cause</code>	4 bits	Halt encoding from <code>kestrel_pkg</code> ; <code>HALT_NONE</code> when running
<code>csr_stub</code>	bit	SYSTEM CSR class: writeback a hard zero

5.2 Keying and Defaults

The truth table is a single unique `casez` keyed on {opcode, funct3, funct7} — a 17-bit key. Every RV32I instruction is distinguished by those three fields alone; the SYSTEM privileged row re-cases on `insn[31:20]` (imm12) to split ECALL/EBREAK/MRET from illegal encodings. Fields that carry immediate bits are keyed as don't-cares (`F7_DC = 7'b???????`) for OP-IMM arithmetic, loads, stores, and branches; the shift rows key funct7 fully, which is what makes mis-encoded shifts illegal rather than silently executed.

Before the case, decode assigns a default bundle: `alu_op = ADD, imm_sel = I`, every control bit clear, `halt_cause = HALT_NONE`. Each row then sets only what it needs. The default: row changes exactly one field, `halt_cause = HALT_ILL` — the entire illegal-instruction mechanism. Every encoding not claimed by a row halts, loudly, instead of executing as something plausible.

5.3 The Decode Table

Transcribed row for row from `rtl/fub/kestrel_decode.sv`. The key column is the 17-bit casez key written as `opcode.funct3.funct7`; any marks a don't-care key field. Bundle columns show only non-default values; `alu` is `alu_op`, `imm` is `imm_sel`, `aPC` is `alu_src_a_pc`, `bImm` is `alu_src_b_imm`, `rd` is `rd_wen`; `dmem` reports `req`, `+we`, and size in bytes (B/H/W); `—` means the default.

5.3.1 Arithmetic and upper-immediate rows

The ALU computes datapath arithmetic here; for LUI the writeback mux bypasses the ALU and takes the immediate directly.

Decode rows — OP, OP-IMM, LUI, AUIPC

opcode.funct3.funct7 (17-bit casez key)	Row	alu	im m	aPC	bImm m	rd
0110011.000.0000000	AD D	AD D	—	—	—	1
0110011.000.0100000	SU B	SU B	—	—	—	1
0110011.001.0000000	SLL	SLL	—	—	—	1
0110011.010.0000000	SLT	SLT	—	—	—	1
0110011.011.0000000	SLT U	SLT U	—	—	—	1
0110011.100.0000000	XO R	XO R	—	—	—	1
0110011.101.0000000	SRL	SRL	—	—	—	1
0110011.101.0100000	SR A	SR A	—	—	—	1
0110011.110.0000000	OR	OR	—	—	—	1
0110011.111.0000000	AN D	AN D	—	—	—	1
0010011.000.any	AD	AD	I	—	1	1

opcode.funct3.funct7 (17-bit casez key)	Row	alu	imm	aPC	bImm	rd
	DI	D				
0010011.010.any	SLT	SLT	I	—	1	1
	I					
0010011.011.any	SLT	SLT	I	—	1	1
	IU	U				
0010011.100.any	XO	XO	I	—	1	1
	RI	R				
0010011.110.any	ORI	OR	I	—	1	1
0010011.111.any	AN	AN	I	—	1	1
	DI	D				
0010011.001.0000000	SLL	SLL	I	—	1	1
	I					
0010011.101.0000000	SRL	SRL	I	—	1	1
	I					
0010011.101.0100000	SR	SR	I	—	1	1
	AI	A				
0110111.any.any	LUI	AD	U	—	1	1
		D				
0010111.any.any	AUI	AD	U	1	1	1
	PC	D				

5.3.2 Loads, stores, branches, jumps

Every branch/jump row uses `alu = ADD` so the ALU produces `pc + imm` for the next-PC mux while the dedicated comparator decides taken-ness in parallel. JALR is the one jump whose ALU left input is `rs1`, not PC.

Decode rows — LOAD, STORE, BRANCH, JAL, JALR

opcode.funct3.funct7 (17-bit casez key)	Row	imm	aPC	bImm	dmem	ctl
0000011.000.any	LB	I	—	1	req, B	—
0000011.001.any	LH	I	—	1	req, H	—

opcode.funct3.funct7 (17-bit casez key)	Row	imm	aPC	bImm	dmem	ctl
0000011.010.any	LW	I	—	1	req, W	—
0000011.100.any	LB U	I	—	1	req, B	—
0000011.101.any	LH U	I	—	1	req, H	—
0100011.000.any	SB	S	—	1	req+ we, B	—
0100011.001.any	SH	S	—	1	req+ we, H	—
0100011.010.any	SW	S	—	1	req+ we, W	—
1100011.000.any	BE Q	B	1	1	—	bra nch
1100011.001.any	BN E	B	1	1	—	bra nch
1100011.100.any	BL T	B	1	1	—	bra nch
1100011.101.any	BG E	B	1	1	—	bra nch
1100011.110.any	BL TU	B	1	1	—	bra nch
1100011.111.any	BG EU	B	1	1	—	bra nch
1101111.any.any	JAL	J	1	1	—	ju mp
1100111.000.any	JAL R	I	—	1	—	jalr

5.3.3 System, fence, and the default

Decode rows — *SYSTEM*, *MISC-MEM*, and the illegal default

opcode	funct3	imm12	Row	Behavior (how the row retires)	halt_cause
111001 1	000	000	EC AL L	halt; RVFI trap beat	4'h1
111001 1	000	001	EB RE AK	halt; RVFI trap beat	4'h2
111001 1	000	302	MR ET	retire as NOP, pc+4 fall-through	—
111001 1	000	other	—	illegal	4'hF
111001 1	001/01 0/011/1 01/110/ 111	any	CS RR W/ S/C + im m for ms	csr_stub: zero readback, writes dropped	—
111001 1	100	any	—	reserved, illegal	4'hF
000111 1	000	any	FE NC E	retire as NOP	—
000111 1	001	any	FE NC E.I	retire as NOP	—
000111 1	010- 111	any	—	reserved, illegal	4'hF
any unclai med	—	—	def aul t	illegal instruction	4'hF

5.4 What the Table Teaches

Three design rulings visible in the table deserve explicit mention:

1. **Reserved encodings halt rather than guess.** Branch funct3 010/011, load/store funct3 011 and 110/111, MISC-MEM funct3 2-7, SYSTEM funct3 100, shift funct7 mismatches: all illegal (unpriv §§2.1.4, 2.1.6 reserve these). A reserved encoding today may become an instruction tomorrow; silently executing it is how compatibility bugs are born.
 2. **The CSR class is one stub row-set, not six special cases.** All six Zicsr forms share `csr_stub = 1`; `rd_wen = 1` and nothing else; the writeback mux owns the zero. The table stays small because the stub is honest about there being nothing to decode.
 3. **Cause 3 appears nowhere in this table.** `HALT_IALIGN` is raised only in `kestrel_core`, which can see the resolved next PC and the branch decision. Decode provably cannot know — a branch's target alignment depends on the register operands it never evaluates.
-

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 kestrel_imm_gen

6.1 Purpose

`kestrel_imm_gen` builds the 32-bit sign- or zero-extended immediate for the five RV32I instruction formats, selected by `imm_sel` from the control bundle. It is pure combinational logic — a five-row case on bit fields of `insn`.

6.2 Interface

kestrel_imm_gen interface

Port	Direction	Width	Description
<code>insn</code>	Input	32	The instruction word (only immediate-bearing bits are used)
<code>sel</code>	Input	<code>imm_sel_e</code>	Format select from decode: I S B U J
<code>imm</code>	Output	32	The assembled immediate

6.3 Internal Structure

The immediate assembly follows the ISA's format definitions exactly (unpriv §2.1):

Immediate assembly per format

<code>sel</code>	Assembly	Formats using it
I	{{20{insn[31]}}}, insn[31:20]}	ADDI/SLTI/SLTIU/XORI/ORI/ANDI, loads, JALR
S	{{20{insn[31]}}}, insn[31:25], insn[11:7]}	stores
B	{{20{insn[31]}}}, insn[7], insn[30:25], insn[11:8], 1'b0}	branches
U	{insn[31:12], 12'b0}	LUI, AUIPC
J	{{12{insn[31]}}}, insn[19:12], insn[20], insn[30:21], 1'b0}	JAL

Notes for maintainers:

- B and J immediates append the implicit low zero (unpriv §2.1) in logic; no consumer may re-shift.
 - U is zero-extended by construction (no sign bits) — LUI writes it directly, AUIPC adds it to the PC.
 - SLTIU's famously surprising semantics (compare against the *sign-extended* immediate interpreted unsigned, unpriv §2.1.4.1) require no special handling here: the immediate is the same sign-extended value every I-format consumer gets; the ALU's unsigned less-than on it preserves the surprise exactly.
-

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

7 kestrel_alu

7.1 Purpose

`kestrel_alu` performs the datapath arithmetic for the core: ten operations selected by `alu_op`, operating on the source-mux outputs. It also computes `eq/lt/ltu` flags — which the core deliberately does not use for branches (see the ruling below).

7.2 Interface

kestrel_alu interface

Port	Direction	Width	Description
<code>op</code>	Input	<code>alu_op_e</code>	Operation select
<code>a</code>	Input	32	Left operand (PC or <code>rs1</code> , via the core's source mux)
<code>b</code>	Input	32	Right operand (immediate or <code>rs2</code>)
<code>y</code>	Output	32	Result
<code>eq / lt / ltu</code>	Output	1 / 1 / 1	Flags: equality, signed less-than, unsigned less-than — unconnected at the core instance

7.3 Operations

ALU operations ($shamt = b[4:0]$)

<code>alu_op</code>	<code>y</code>	Notes
ADD	$a + b$	Also computes branch/JAL targets when $a = pc$
SUB	$a - b$	
AND / OR / XOR	bitwise	
SLL	$a \ll shamt$	$shamt = b[4:0]$ (unpriv §2.1.4.1: shift amounts use 5 bits)
SRL	$a \gg shamt$	logical

alu_op	y	Notes
SRA	<code>\$signed(a) >>> shamt</code>	arithmetic
SLT	<code>{{31{1'b0}}}, lt</code>	sets rd to 0/1
SLTU	<code>{{31{1'b0}}}, ltu</code>	unsigned compare, notably for SLTIU's sign-extended-then-unsigned immediate

The shift-by-b[4:0] matches the ISA's shamt semantics exactly — and because the decode table keys SLLI/SRLI/SRAI fully on funct7, encodings with shamt[5] set (reserved in RV32I) never reach this module as shifts; they halt as illegal in decode.

7.4 The Flags Ruling

The flags compare a against b: `eq = (a == b)`, `lt = ($signed(a) < $signed(b))`, `ltu = (a < b)`. For branch instructions the core drives `a = pc` (the ALU is simultaneously computing `pc + imm`), so these flags would compare pc against imm — silently wrong. The core's instance leaves `eq/lt/ltu` unconnected and branches decide on the dedicated rs1/rs2 comparator in the top. The ALU flags remain correct for SLT/SLTU's internal use (`lt/ltu` feed the SLT/SLTU rows) and are available to any future consumer whose a/b are the intended operands. This was a reviewed ruling (Task 4 review, SDD ledger), recorded here so nobody “fixes” the dangling pins.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 kestrel_regfile

8.1 Purpose

kestrel_regfile is the 32 x 32-bit general-purpose register file: two combinational read ports, one synchronous write port, and the architectural x0 rule enforced in hardware. Reads are combinational because the single-cycle design cannot afford a read latency; the write commits on the clock edge.

8.2 Interface

kestrel_regfile interface

Port	Direction	Width	Description
clk / rst_n	Input	1 / 1	Clock; active-low reset clears all 32 entries
rs1_addr / rs1_data	Input / Output	5 / 32	Read port 1 (combinationally regs[rs1_addr])
rs2_addr / rs2_data	Input / Output	5 / 32	Read port 2
rd_addr / rd_data / rd_wen	Input / Input / Input	5 / 32 / 1	Write port; committed on the edge when rd_wen and rd_addr != 0

8.3 Internal Structure and Rules

- Storage is logic [31:0] regs [32], reset to all zeros (so no register carries architectural garbage out of reset, and pre-load observability is deterministic).
- The x0 rule lives here:** writes with rd_addr == 0 are discarded by the write condition rd_wen && (rd_addr != X0_ADDR); reads of x0 return the reset/stored zero naturally.
- The core additionally gates rd_wen (into rd_wen_eff) so that the retry's first beat and halted cycles never reach this port — see the kestrel_core chapter.
- No write-forwarding exists: a write commits on the edge and is readable the next cycle. This is exactly the single-cycle contract (a store is visible to the next cycle's load *through the memory*; a register write is visible to the next instruction *through the register file*).

Last Updated: 2026-10-07

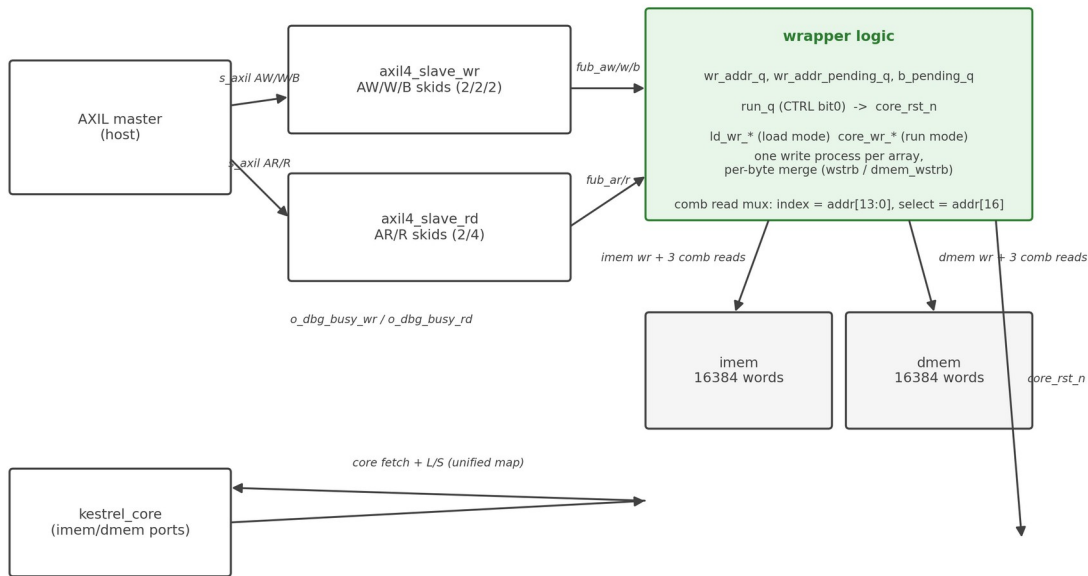
RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 kestrel_mem_loader

9.1 Purpose

`kestrel_mem_loader` is the optional board glue (HAS Ch. 4 specifies its external contract; this section covers how it is built). It contains: two 16384-word distributed-RAM arrays (`imem`, `dmem`), a CTRL/run register, and the write-port merge logic — wrapped in the repo's skid-buffered AXIL leaf slaves, composed exactly as `rtl/amba/shared/sdpram_slave_axil_axil.sv` composes them. Mutual exclusion between loader and core is by protocol; there is no contention arbiter and no FSM in the datapath.

9.1.1 Figure 2.1: Loader composition



kestrel_mem_loader internal composition

9.2 Memory Arrays

- `(* ram_style = "distributed" *) logic [31:0] imem [0:MEM_WORDS-1]` (and `dmem`) — the attribute keeps reads combinational, which the single-cycle contract requires; a block RAM's synchronous read would break it.
- One write port each, merging two clients per byte lane in a single clocked process: the loader (`ld_wr_*`, gated by `!run_q`) and the core (`core_wr_*`, gated by `run_q && dmem_req && dmem_wstrb != 0`). Both are additionally fenced during reset assertion.

- Under the unified map each array carries up to three combinational read ports (core fetch, core L/S, AXIL readback); distributed-RAM read replication absorbs that in synthesis.
- Power-up content is undefined in hardware; the simulation model zero-initializes so pre-load reads are deterministic. An image is always loaded before run.

9.3 CTRL / Run Register and Reset Sequencing

`run_q` is a sticky bit (set by a CTRL write with `wstrb[0]` & `wdata[0]`, cleared only by reset). `core_rst_n` is literally `run_q`: low in load mode (the core is held, parks its fetch at `RESET_ADDR`, retires nothing), high from the commit cycle of the run write. Power-on default is load mode.

9.4 Write Side: Leaf Slave plus Two Holding Bits

The `axil4_slave_wr` leaf terminates the AW/W/B protocol into FUB nets. The wrapper's entire write-side state is two holding bits plus the run register:

- `wr_addr_q` / `wr_addr_pending_q`: the AW address holds until its W beat fires (`fub_awready = !wr_addr_pending_q || w_fire`; a same-cycle swap lets a new AW in as the old W fires).
- `b_pending_q`: at most one B outstanding (`fub_wready = wr_addr_pending_q && !b_pending_q`); B returns OKAY one holding bit deep and the leaf's B skid buffers it against the master.

Loader-write qualification: `ld_wr_fire = w_fire && !wr_addr_q[CTRL_BIT] && !run_q` — memory writes land only in load mode; CTRL writes never touch the arrays. Run-mode loader writes are dropped in logic with B still OKAY (testplan scenario ML-09).

9.5 Read Side: Combinational Beat through the AR Skid

The `axil4_slave_rd` leaf supplies the AR/R handshake; the wrapper's response is a single-cycle combinational beat: `fub_arready = fub_rready` (AR accepted exactly when the R skid can take the beat), `fub_rvalid = fub_arvalid`, `fub_rresp = OKAY`. The read mux is:

```
fub_rdata = fub_araddr[CTRL_BIT] ? {31'b0, run_q}
           : fub_araddr[DMEM_BIT] ? dmem[fub_araddr[WORD_IDX_W-1:0]]
           : imem[fub_araddr[WORD_IDX_W-1:0]];
```

Core-side reads select the array the same way: `imem_rdata = imem_addr[DMEM_BIT] ? dmem[imem_addr[13:0]] : imem[imem_addr[13:0]]` (and likewise for `dmem_rdata`), which is what makes one flat image coherent across fetch, L/S, and AXIL readback.

9.6 Address Map, as Implemented

Loader map implementation facts

Item	Implementation
Array word index	<code>addr[13:0]</code> of the presenting port's byte address (<code>WORD_IDX_W = \$clog2(MEM_WORDS) = 14</code>), AXIL and core alike
Array select	<code>addr[16]: 0 = imem, 1 = dmem</code> , on every port (unified map)
CTRL decode	<code>addr[17] = 1</code> → CTRL register at <code>0x0002_0000</code>

Two consequences the maintainer must not “tidy” without a plan-level decision: byte-address bits [15:14] are not decoded, so each region's 64 KB byte window aliases onto its array every `0x4000` bytes; and because the core presents only word-aligned addresses, its window is the 4096 word-aligned slots (16 KB of image) per array — the delivery testbench streams word k to AXIL byte address $4*k$, which is the convention that keeps loader and core views identical. The module header's `addr[15:2]` wording describes intent; the implemented `addr[13:0]` slice above is the as-built truth this specification documents.

9.7 Debug Taps

`o_dbg_busy_wr` / `o_dbg_busy_rd` are the leaf slaves' busy outputs, exposed for the AXIL backpressure checks (scenario ML-08).

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

10 Package Types and Top-Level Wiring

10.1 kestrel_pkg

`kestrel_pkg` (`rtl/includes/kestrel_pkg.sv`) is the shared type and encoding source. It contains no state and no modules; every `kestrel` filelist pulls it in with `-f`, and consumers never name the package path directly.

10.1.1 ALU operation enum (`alu_op_e`, 4 bits)

alu_op_e enumerators

Enumerator	Operation
ADD SUB	add / subtract
AND OR XOR	bitwise
SLL SRL SRA	shifts (<code>shamt = b[4:0]</code>)
SLT SLTU	set-less-than, signed / unsigned

10.1.2 Immediate-format enum (`imm_sel_e`, 3 bits)

imm_sel_e enumerators

Enumerator	Format	Assembly
I	I-type	{20{ <code>insn[31]</code> }}, <code>insn[31:20]</code>
S	S-type	{20{ <code>insn[31]</code> }}, <code>insn[31:25]</code> , <code>insn[11:7]</code>
B	B-type	{20{ <code>insn[31]</code> }}, <code>insn[7]</code> , <code>insn[30:25]</code> , <code>insn[11:8]</code> , 1'b0}
U	U-type	{ <code>insn[31:12]</code> , 12'b0}
J	J-type	{12{ <code>insn[31]</code> }}, <code>insn[19:12]</code> , <code>insn[20]</code> , <code>insn[30:21]</code> , 1'b0}

10.1.3 Halt-cause encodings (`localparam logic [3:0]`)

HALT_ encodings — the single source of truth shared by decode and core*

Name	Value	Raised in	Meaning
HALT_NONE	4'h0	—	running
HALT_ECALL	4'h1	<code>kestrel_decode</code>	ECALL

Name	Value	Raised in	Meaning
HALT_EBREAK	4'h2	kestrel_decode	EBREAK
HALT_IALIGN	4'h3	kestrel_core	taken control transfer to a non-4-aligned target
HALT_ILL	4'hF	kestrel_decode	illegal encoding (default row)

The package comment records the ownership split: causes 1, 2, and F belong to decode; cause 3 belongs to the core because the condition needs the resolved next PC and branch decision that decode cannot see.

10.2 Top-Level Wiring (kestrel_core)

The instance map every maintainer should be able to redraw from memory:

kestrel_core internal wiring map

Net(s)	Driver → Load	Notes
insn = imem_rdata	fetch → decode, imm_gen, regfile addrs, RVFI	opcode = insn[6:0], funct3 = insn[14:12]
Control bundle (13)	u_decode → muxes, regfile, L/S datapath	keyed on {opcode, funct3, funct7}
imm	u_imm_gen → alu_src_b mux, writeback (LUI)	
rs1_data / rs2_data	u_regfile → source muxes, comparator, store-data rotators	read addrs from insn[19:15] / insn[24:20]
alu_y	u_alu → dmem address, writeback mux	ALU flags unconnected (ruling, kestrel_alu chapter)
cmp_eq/cmp_lt/ cmp_ltu →	core comparator → next-PC mux	funct3 selects

Net(s)	Driver → Load	Notes
branch_cond → branch_taken next_pc		the condition
	next-PC mux → PC register, rvfi_pc_wdata	4-row priority mux
rd_wdata / rd_wen_eff	writeback mux + gates → u_regfile	x0 discard inside the file
L/S nets (ls_*, dmem_*)	core rotators → dmem pins	retry state ls_retry, merge register ls_rdata_lo
Halt nets (halt_*, misalign_target)	core → halt ports, halt_q	halt_cause_eff = dec_halt_cause \\ (misalign_targ et ? HALT_IALIGN : HALT_NONE)
RVFI bundle (17)	core aggregation assigns → top ports	rules in Chapter 4

10.3 Reset Discipline

All sequential logic uses the shared `reset_defs.svh` macros (`ALWAYS_FF_RST`, `RST_ASSERTED`): asynchronous assert, synchronous deassert, active-low `rst_n`. The register file clears to zeros; the PC clears to `RESET_ADDR`; `halt_q`, `ls_retry`, `ls_rdata_lo`, and `retire_count` clear to zero. The loader applies the same macros with `aresetn`, and its `core_rst_n` output is the core's actual reset in loader integrations.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 The Single Cycle, Step by Step

11.1 The Clock in One Sentence

On every rising edge, kestrel writes one register-file entry (maybe), updates the PC (unless held), and moves its bookkeeping flops — while a combinational cloud standing between pc and the next edge computes everything the next cycle needs: the fetched instruction, its decode, the operands, the result, the memory access, the next PC, and the RVFI record for whatever retires this cycle.

The sections below follow the data in dependency order through a fully general cycle.

11.2 1. Fetch and Decode

`imem_addr = pc` and `insn = imem_rdata` — the instruction memory answers in the same cycle. Decode is the truth table of the `kestrel_decode` chapter, producing the thirteen-field control bundle plus defaults of “no operation, no halt.”

11.3 2. Operands and Immediates

The register file reads `rs1 = insn[19:15]` and `rs2 = insn[24:20]` combinationally; `x0` reads zero because the file ties it. The immediate generator builds one of the five format immediates under `imm_sel`. Two muxes select the ALU inputs: `alu_src_a` is the PC (for AUIPC and branch targets) or `rs1_data`; `alu_src_b` is the immediate or `rs2_data`.

11.4 3. Execute

The ALU computes one of ten operations. For branches, the dedicated comparator evaluates `cmp_eq/cmp_lt/cmp_ltu` on `rs1_data/rs2_data` in parallel, with `funct3` selecting the condition in a unique case. Both the ALU’s target (`pc + imm`, since `alu_src_a = pc` on branch rows) and the taken decision are ready when the next PC is chosen — taken and not-taken branches cost the same cycle.

11.5 4. Memory Access

For loads and stores, `dmem_addr` is the word-aligned ALU result (`{alu_y[31:2], 2'b00}` — or the next word on a retry beat), `dmem_wdata/dmem_wstrb` are the rotated store data and byte enables, and read data is rotated back before size/sign selection implements LB/LBU/LH/LHU/LW. Aligned and in-word-misaligned accesses complete in this one cycle; a cross-word access holds the PC for a retry cycle and issues a second beat (next section).

11.6 5. Writeback

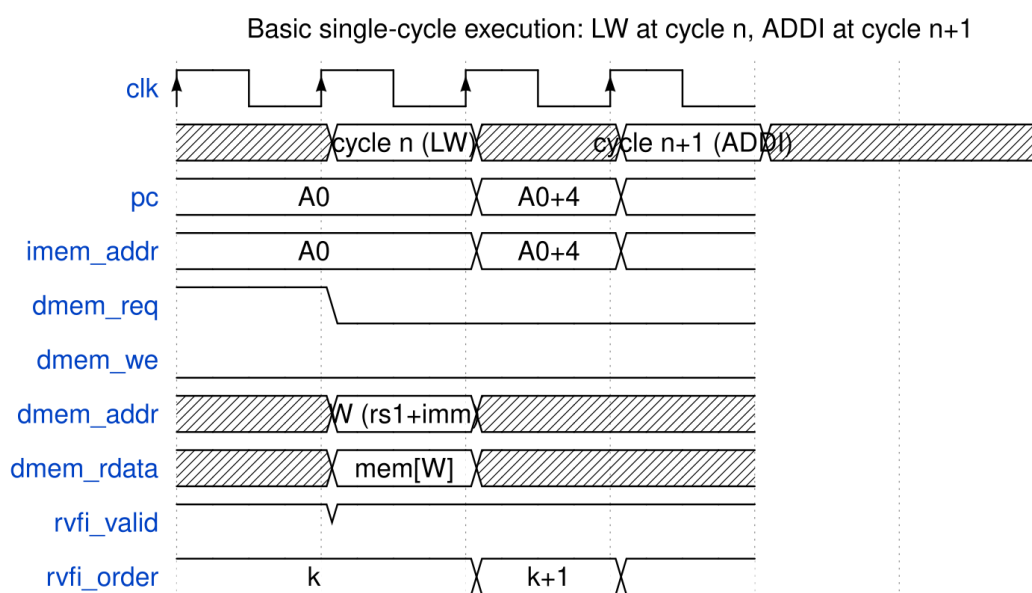
`rd_wdata` is selected by the five-row priority mux (CSR stub zero → LUI immediate → load data → link `pc+4` → ALU result); `rd_wen_eff = rd_wen & ~ls_first & ~halt` qualifies it; the write commits on the edge.

11.7 6. Next-PC Selection and the Edge

The next-PC mux selects among JAL/JALR target, taken-branch target, and `pc + 4`. The PC register loads `next_pc` on the edge — unless `halt` (frozen) or `ls_first` (held for the retry beat). `rvfi_pc_wdata = next_pc` reports the architecturally next PC on the retire beat.

11.8 The Basic Cycle on the Pins

11.8.1 Waveform 4.1: Basic single-cycle execution — an aligned load followed by an ALU op



Waveform 4.1: basic single-cycle execution — an aligned load followed by an ALU op

Two back-to-back retiring instructions. During cycle n the LW's full path — imem read, decode, register read, ALU address, dmem read, rotate/sign-extend, writeback mux — completes between the same two edges, and the beat retires (`rvfi_valid=1`) with the load's memory fields. The ADDI in cycle $n+1$ retires just the same: CPI 1 with no bubbles, because the whole machine is combinational between the PC and the edge.

11.9 Branch and Jump Resolution

Control-transfer resolution

Transfer	Target computation	Decision source	Cost
BEQ/BNE/ BLT/BGE/ BLTU/BGEU	$pc + imm(B) \text{ — ALU row}$ $alu_src_a_pc=1$	dedicated comparator, funct3-selected	1 cycle, taken or not
JAL	$pc + imm(J)$	unconditional (jump=1)	1 cycle
JALR	$(rs1_data + imm) \&$ $32'hFFFFFF_FFFE$	unconditional (jalr=1)	1 cycle
Taken transfer to next_pc[1:0] != 00	computed but not committed	misalign_target → halt cause 0x3	halts (next section)

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 Cross-Word Misaligned Retry

12.1 When the Retry Fires

The L/S datapath completes any access with $\text{addr}[1:0] + \text{size} \leq 4$ in the single cycle — every aligned access and every misaligned access that stays inside one word. When $\text{addr}[1:0] + \text{size} > 4$ the access straddles two words, one 32-bit beat cannot carry it, and the retry machinery engages:

- $\text{ls_crossing} = \text{ls_active} \ \& \ ((\text{ls_offset} + \text{ls_size_bytes}) > 4)$
- $\text{ls_first} = \text{ls_crossing} \ \& \ \sim \text{ls_retry}$ — the first beat is in flight
- $\text{ls_second} = \text{ls_retry}$ — the retry beat
- $\text{ls_retry} \leq \text{ls_first}$ — one bit of control state; no FSM

12.2 Beat-by-Beat Behavior

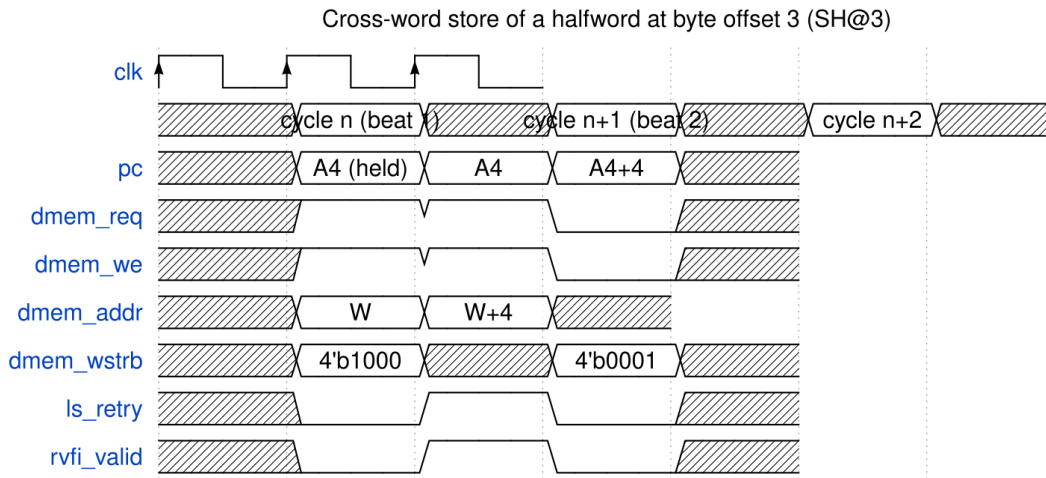
The two beats of a cross-word access ($o = \text{addr}[1:0]$)

Aspect	Beat 1 (ls_first)	Beat 2 (ls_second)
dmem_addr	$\{\text{alu_y}[31:2], 2'b00\}$ — word containing the start	$\{\text{alu_y}[31:2] + 1, 2'b00\}$ — the next word
Store data	$\text{rs2_data} \ll (8*o)$	$\text{rs2_data} \gg (8*(4-o))$
Store strobe	$\text{size_mask} \ll o$ (tail bytes)	ls_head_strobe (head bytes only)
Load path	$\text{ls_rdata_lo} \leq \text{dmem_rdata} \gg (8*o)$	$\text{ls_raw_rdata} = (\text{dmem_rdata} \ \& \ \text{head_mask}) \ll (8*(4-o)) \ \vee \ (\text{ls_rdata_lo} \ \& \ \text{tail_mask})$
PC	$\text{held}(\text{pc} \leq \text{ls_first} ? \text{pc} : \text{next_pc})$	advances on the edge
rvfi_valid	low — no partial beat leaks	high — the access retires once, with assembled data

Everything else in the core is unaware the retry happened: dmem_req (and dmem_we for stores) simply stays high across both beats while the address steps by one word.

12.3 The Waveform

12.3.1 Waveform 4.2: Cross-word store retry — SH at byte offset 3



Waveform 4.2: cross-word store retry — SH at byte offset 3

A store-halfword at byte offset 3 (the SH@3 case): beat 1 writes the tail byte of word W with strobe 4'b1000; beat 2 writes the head byte of word W+4 with strobe 4'b0001, data right-rotated by the tail size. dmem_req/dmem_we stay high across both beats, ls_retry marks the retry cycle, the PC holds for one extra cycle, and the instruction retires exactly once — rvfi_valid high only on the final cycle.

12.4 Worked Example: a Halfword Across the Boundary

lh x2, 7(x10) with x10 = 0x1000 reads the halfword at 0x1007 — one byte at offset 3 of word 0x1000 and one at offset 0 of word 0x1004.

- Beat 1: dmem_addr = 0x1000, read data >> 24, captured into ls_rdata_lo — the tail byte.
- Beat 2: dmem_addr = 0x1004, head byte merged at bits [15:8]; the halfword is sign-extended from bit 15 into x2.
- RVFI: one beat, mem_addr = 0x1007, rmask = 0011 packed from the addressed address, mem_rdata holding the assembled halfword in bits [15:0].

A directed test with distinct nonzero bytes on both sides of the boundary (ls_misaligned) pins this end to end; a first-cut implementation that silently truncated bytes rotating past bit 31 reported 0x00FE where the architectural answer was 0xCAFE — caught because the golden interpreter models the full two-word semantics and the lockstep diff tolerates no slop.

12.5 Why Hardware, Why Two Cycles

Trapping to software emulation needs the trap machinery kestrel deliberately does not have, and emulating in a trap stub would make the misaligned case a hundred cycles instead of two. Hardware handling keeps every access at fixed cost — one cycle in-word, two crossing — which is the only cost model a single-cycle core can afford to reason about. The price is integration: a misaligned word access performs two word reads/writes and is not atomic, which the spec never promised anyway (unpriv §2.1.6). Instruction-side misalignment is not permitted this latitude — the next section covers the halt.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 Halt, IALIGN, and the System Layer

13.1 The Halt Holding Register

halt_q is a one-bit flop that latches any nonzero halt_cause_eff:

```
halt_cause_eff = dec_halt_cause | (misalign_target ? HALT_IALIGN :  
HALT_NONE)  
halt_now      = |halt_cause_eff  
halt_q        <= halt_q | halt_now  
halt          = halt_q | halt_now      // visible on the first  
halting cycle
```

Decode raises HALT_ECALL (1), HALT_EBREAK (2), and HALT_ILL (F); the core raises HALT_IALIGN (3). Once latched:

- The PC freezes (pc <= halt ? pc : ...).
- Writeback is suppressed (rd_wen_eff and the RVFI rd_wb gate both use halt).
- Retirement stops: rvfi_valid = rst_n & ~halt_q & ~ls_first, and since halt_q latches on the halting cycle, the halting instruction itself is the last beat — a trap beat with rvfi_trap = halt_now.

A holding register rather than a pulse matters for verification: an external observer sampling any time after the halt sees a stable, unambiguous stopped state with the cause still on the port. The testbench samples four post-halt cycles pinning halt raised, rvfi_valid low, rvfi_pc_rdata frozen.

13.2 The IALIGN Halt (Cause 3)

With IALIGN=32 — no compressed instructions — every fetch must be 4-byte aligned, and unpriv §2.1.5 requires an *instruction-address-misaligned* exception when taken control flow targets an address whose low two bits are not 00. RISC-V's permission to handle *data* misalignment in hardware does not extend here.

The condition, visible only in the core:

```
misalign_target = (jump | jalr | branch_taken) & (next_pc[1:0] !=  
2'b00)
```

Why decode cannot catch it: for JALR the target depends on a register value; for every branch it depends on the taken decision. Decode sees neither — it produces the control bundle, not the data. kestr_el_pkg states the ownership directly: cause 3 belongs to the core.

The writeback subtlety. JAL/JALR set decode's rd_wen — they are link instructions, and a taken JAL/JALR that *does* align must write pc+4. The misaligned case must not. Rather than teaching

decode about alignment, the core gates both the register write and the RVFI report with `halt` itself: `rd_wen_eff = rd_wen & ~ls_first & ~halt`, `rd_wb = rd_wen & (insn[11:7] != 0) & ~halt`. The trap beat reports the misaligned target on `rvfi_pc_wdata` (it is `next_pc`), exactly what a real trap would record as the faulting address — a detail riscv-formal's checks verify explicitly.

13.3 The System Layer: Stubs, NOPs, and Halts

The thirty-seven base instructions sit inside two opcode classes that belong to the wider system: MISC-MEM (FENCE and friends, unpriv §2.1.7; FENCE.I is Zifencei, §4.1) and SYSTEM (ECALL, EBREAK, and the Zicsr CSR instructions, §4.2; MRET is defined in the privileged Volume II). *kestrel* has no memory-ordering hardware, no CSR file, and no trap handler, so these classes retire through documented stubs and NOPs. Exactly what that means — including what it does *not* mean:

13.3.1 FENCE and FENCE.I retire as NOPs

kestrel's `imem` and `dmem` are separate, combinational-read ports on a single-cycle core: there is no write buffer to drain, no cache to invalidate, no instruction prefetch to refetch. Fence semantics are vacuous at this rung, and decode retires FENCE (MISC-MEM funct3 000) and FENCE.I (funct3 001) as NOPs — one cycle each, no architectural effect. MISC-MEM funct3 2-7 are reserved and halt as illegal. (The one battery test whose program modifies its own instruction stream, `rv32ui-p-fence_i`, works because the *testbench memory* is a single unified array serving both ports — a testbench coherence choice, not a core feature.)

13.3.2 MRET falls through

MRET (SYSTEM funct3 000, imm12 0x302) returns from a machine trap. It is meaningful only when traps exist — trap state, `mepc`, privilege transfers, all rung-3 machinery. *kestrel* retires MRET as a documented NOP-with-fall-through: the PC advances to `pc+4`. That choice is not arbitrary: the riscv-tests p-environment points `mepc` at the instruction following its trap stub in the configurations *kestrel* runs, so `pc+4` is exactly what the test expects. This is stated plainly so nobody mistakes it for trap support.

13.3.3 The CSR class retires as a bounded stub

The six Zicsr forms (SYSTEM funct3 001/010/011/101/110/111) retire through `csr_stub`: decode raises `csr_stub` and `rd_wen`, and the writeback mux substitutes a hard zero. Reads return zero; writes drop on the floor. SYSTEM funct3 100 is reserved and halts as illegal.

Say it plainly: **this is not CSR support**. There is no CSR file, no `mstatus`, no `mtvec`, no counters, and nothing the instructions do has architectural effect beyond a zero written to `rd`. The stub exists for one bounded reason: the riscv-tests p-environment preamble executes about a dozen CSR instructions plus an MRET before every test body, and without the stub the battery cannot run at all. On the battery programs the only CSR read with `rd != x0` is `mhartid` (value 0), which the stub matches. Anything needing real CSR state — trap handlers, counters, feature probing — is out of contract at this rung and fails loudly either as a zero where a nonzero was expected or as a

program that halts where it should have trapped. The honesty matters for the ladder: merlin inherits the same stub, and peregrine replaces it with a real machine-mode CSR file. The seam is deliberately visible in the decode table.

13.4 Halt Timing Summary

Halt timing at the boundary

Cycle	halt	rvfi_valid	rvfi_trap	PC	Notes
Last running cycle	0	1	0	advanc	normal retirement
Halting cycle	1 (immediate)	1	1	frozen from this edge	trap beat; cause on halt_cause; no rd/mem fields
All later cycles	1	0	0	frozen	halt_q holds; cause held on the port

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

14 RVFI Aggregation

14.1 Retirement as the Observation Channel

Every instruction that retires produces one beat; the halting instruction produces one final trap beat; nothing else produces anything. The cocotb testbench diffs these beats field by field against a golden interpreter, and riscv-formal relates them to an instruction-set model. Both consumers need the aggregation rules below to be exact — they are stated next to the assignments in the RTL and restated here.

14.2 The Field Table

RVFI signals and their kestrel aggregation

Signal	Width	kestrel aggregation
rvfi_valid	1	rst_n & ~halt_q & ~ls_first — high exactly on retiring cycles
rvfi_order	64	retire_count; 0, 1, 2, ... over retired beats
rvfi_pc_rdata	32	The retiring instruction's PC
rvfi_pc_wdata	32	next_pc — the architecturally next PC (the misaligned target on a cause-3 halt)
rvfi_insn	32	The instruction word
rvfi_trap	1	halt_now: high on the single halting trap beat
rvfi_rs1_addr / rvfi_rs2_addr	5+5	Register-file read ports (insn[19:15], insn[24:20])
rvfi_rs1_rdata / rvfi_rs2_rdata	32+32	Read data; x0 reads zero naturally
rvfi_rd_addr / rvfi_rd_wdata	5+32	Writeback, when rd_wb; both zero otherwise
rvfi_mem_addr	32	The unaligned byte address of a load/store (alu_y); zero otherwise
rvfi_mem_rmask / rvfi_mem_wmask	4+4	Byte lanes touched, packed from bit 0 relative to rvfi_mem_addr (a word access reports 4'b1111 regardless of offset)
rvfi_mem_rdata / rvfi_mem_wdata	32+32	Assembled access data (loads) / size-truncated store data (rs2_data & ls_data_mask); zero otherwise

14.3 The Three Gating Rules

Three qualifications turn the raw datapath signals into a clean retire stream:

1. **x0 reports zero.** $rd_wb = rd_wen \ \& \ (insn[11:7] \neq 0) \ \& \ \sim halt$, so a beat writing x0 reports $rd_addr = 0$ with $rd_wdata = 0$. This is not merely tidiness: riscv-formal requires $rd_wdata == 0$ whenever $rd_addr == 0$, and the early core failed exactly there until the rule was pinned. The register file independently discards the x0 write architecturally.
2. **The retry's first cycle is silent.** During the first beat of a cross-word access, $rvfi_valid$ is low (ls_first masks it): no partial beat leaks out. The access reports one beat on its final cycle, with the full assembled data.
3. **Halt retires one trap beat, then silence.** $rvfi_trap = halt_now$ marks the halting instruction itself — a beat with trap set, no rd write, no memory fields — after which the latched $halt_q$ holds $rvfi_valid$ low forever. The wrapper used by riscv-formal derives $rvfi_halt = rvfi_valid \ \& \ halt$, which matches the semantics exactly: the trap beat is the final retirement. ($rvfi_intr = 0$, $rvfi_mode = M$, $rvfi_ixl = 32$ are generated in the wrapper, not the core.)

14.4 Memory-Field Packing

For loads and stores, $rvfi_mem_addr$ is the *unaligned* byte address and the masks count byte lanes from bit 0 relative to that address — the riscv-formal byte-*i* convention of $rvfi.md$. A store's $rvfi_mem_wdata$ is the store value truncated to the access size ($rs2_data \ \& \ ls_data_mask$), not the raw register; leaking the upper bytes into the payload is a real bug class this encoding exists to prevent, and one kestrel fixed when the battery's ma_data -shaped vectors arrived. For a cross-word access the single retire beat reports the full-size mask with the assembled data — the retry's two physical beats collapse into one architectural record.

14.5 Why This Pays

Because the rules are few and stated in RTL comments next to the assignments, the golden interpreter models the same rules in Python and the two can be diffed beat for beat over tens of thousands of instructions with zero tolerance. When a diff appears, it names the cycle, the PC, and the field — which is how the misaligned-jump defect (found by riscv-formal as a counterexample the 42-test battery missed) was localized in minutes rather than debugged from waveforms.

Last Updated: 2026-10-07

15 Verification Hooks and Observability

15.1 The Three Testplan-Tracked Hooks

kestrel was built RVFI-first, and the verification architecture hangs three observable hooks off the boundary. Each maps to committed testplan scenarios:

The three testplan-tracked hooks

Hook	Signals	What it proves	Testplan anchor
RVFI retire channel	17 rvfi_* ports	Architectural correctness: every retired beat diffed full-field against the golden interpreter rv32i_interpreter.py (pc, insn, order, trap, rs/rd, all four memory channels — no sampled subset, no tolerance); riscv-formal's 42 checks relate the same beats to ISA models	kestrel_core CORE-01..CORE-05, CORE-14; kestrel_mem_loader ML-07
Halt observability	halt, halt_cause	Program end, cause classification, and the post-hold: the halting trap beat, then at least four cycles of halt raised with rvfi_valid low and the PC frozen	kestrel_core CORE-12, CORE-13
Data-port watch	dmem_req, dmem_addr, dmem_wstrb, dmem_wdata	The memory contract on the pins: rotated strobes land the right bytes (TB per-byte merge), retry addressing steps one word, the tohost mailbox is observed on the store port, store-to-load	kestrel_core CORE-08..CORE-11; loader ML-02/ML-05/ML-06

Hook	Signals	What it proves	Testplan anchor
		visibility is single-cycle	

15.2 Supporting Hooks

Supporting loader hooks

Hook	Signals	Use
Loader reset tap	core_rst_n	Load-mode isolation: with rst_n released but CTRL . run still 0, the core is held, its PC parks at RESET_ADDR, and loader writes become visible on the core-side read port (ML-04)
Leaf busy taps	o_dbg_busy_wr, o_dbg_busy_rd	AXIL backpressure: B/R must hold stable under stalled bready/rready with busy asserted (ML-08)
Run-mode write rejection	AXIL write + readback	A loader write in run mode is dropped in logic with B OKAY — the halted core's ECALL word must not be poisoned (ML-09)

15.3 The Consumer Stack

Verification consumer stack

Layer	Mechanism	Scope
Golden interpreter lockstep	rv32i_interpreter.py emits the same RVFI beat format as the core; the cocotb harness diffs full-field, zero tolerance	Every golden-trace test, plus the constrained-random fuzz (5 gate / 25 func / 200 full streams)
Spike lockstep	spike --isa=RV32I -l commit stream walked 1:1 against the core's (pc, insn) beats; exception records verified per beat; exit code confirms tohost==1	Battery and fuzz at func level, using the hardware-misaligned spike 1.1.0 build
rv32ui battery	42 self-checking riscv-tests programs run to the ECALL halt; verdict = x3's last writeback	3/3 gate, 42/42 func and full
riscv-formal	42 bounded-model-checking proofs (37 instruction models + reg/pc_fwd/pc_bwd/liveness/unique) against the RVFI port, depth 6, z3/smtbmc	Every RTL change; the gate that caught the misaligned-jump bug 42 directed tests missed

15.4 Cross-Reference to the Testplan Set

The six plans in `dv/testplans/` are the V&V cross-reference for both this MAS and the HAS: `kestrel_core_testplan.yaml` (18 scenarios — ISA classes, branches, loads/stores incl. the cross-word retry, system layer, battery, lockstep, fuzz, coverage closure at 94.9% line), `kestrel_mem_loader_testplan.yaml` (ML-01..ML-09), and per-FUB plans (`kestrel_alu`, `kestrel_decode`, `kestrel_imm_gen`, `kestrel_regfile`) whose directed rows pin each block's unit behavior against the chapters of this document. Coverage closure is measured across all cells (Verilator line coverage through the central `cov_utils` hook); regenerate with the commands recorded in each plan.

Last Updated: 2026-10-07

16 Parameters, Defines, and Filelists

16.1 Parameters

16.1.1 kestrel_core

kestrel_core parameters

Parameter	Type	Default	Verified values	Description
RESET_ADDR	logic [31:0]	32'h0000_ 0000	0x0, 0x100, 0x8000_0000	Reset fetch address; the first retired beat's pc_rdata equals it (testplan scenario CORE- 03)

16.1.2 kestrel_mem_loader

kestrel_mem_loader parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	18	AXIL slave address width; bits [17:0] of the map
AXIL_DATA_WIDTH	int	32	One 32-bit word per beat
MEM_WORDS	int	16384	Words per array (64 KB imem + 64 KB dmem)
SKID_DEPTH_AW / SKID_DEPTH_W / SKID_DEPTH_B	int	2 / 2 / 2	axil4_slave_wr skid depths
SKID_DEPTH_AR / SKID_DEPTH_R	int	2 / 4	axil4_slave_rd skid depths

Only the default loader parameterization is exercised by the testplans (single build config; recorded as verified, no sweep claimed). RESET_ADDR coverage is recorded per value in the core testplan.

16.2 Defines and Includes

Shared defines

Include	Used by	Purpose
reset_defs.svh (via -f \$REPO_ROOT/rtl/amba/f ilelists/reset_defs.f)	kestrel_core, kestrel_regfile, kestrel_mem_loader	ALWAYS_FF_RST / RST_ASSERTED macros: asynchronous assert, synchronous deassert, active-low

No define configuration surface exists beyond this; there are no feature ifdefs in the kestrel RTL.

16.3 Filelists

Filelists (KESTREL_ROOT = projects/components/riscv-ip/kestrel-rv32i; REPO_ROOT = RTLDesignSherpa checkout root)

Filelist	Contents	Needs
rtl/filelists/kestrel_pkg.f	kestrel_pkg alone	\$KESTREL_ROOT
rtl/filelists/fub/kestrel_alu.f	ALU + package	\$KESTREL_ROOT
rtl/filelists/fub/kestrel_decode.f	Decode + package	\$KESTREL_ROOT
rtl/filelists/fub/kestrel_imm_gen.f	Immediate generator + package	\$KESTREL_ROOT
rtl/filelists/fub/kestrel_regfile.f	Register file + reset_defs + package	\$KESTREL_ROOT, \$REPO_ROOT
rtl/filelists/fub/kestrel_mem_loader.f	Loader + reset_defs + package + axil4_slave_wr/axil4_slave_rd closures	\$KESTREL_ROOT, \$REPO_ROOT
rtl/filelists/top/kestrel_core.f	Core top + all core FUB closures	\$KESTREL_ROOT, \$REPO_ROOT
rtl/filelists/kestrel_all.f	Master closure: everything above	\$KESTREL_ROOT, \$REPO_ROOT
dv/filelists/kestrel_tb.f	kestrel_tb_top + kestrel_loader_tb_top on kestrel_all.f	\$KESTREL_ROOT

Compile-order and closure discipline: each list is complete for its module (package, includes, leaf dependencies); `kestrel_all.f` includes everything with `-f` and never hand-lists sources. The filelists are the supported compile interface.

16.4 Regenerating This Document

```
# from projects/components/riscv-ip/kestrel-rv32i/docs
./generate_mas_pdf.sh --rev 0.1      # KESTREL_MAS_v0.1.docx / .pdf
./generate_has_pdf.sh --rev 0.1      # KESTREL_HAS_v0.1.docx / .pdf
python3 gen_kestrel_figures.py      # regenerate figures + waveform
PNGs
```

`gen_kestrel_figures.py` (this directory) redraws the block/composition figures with matplotlib and renders the wavedrom sources in `kestrel_mas/assets/wavedrom/*.json` via `wavedrom-cli + inkscape`; both build scripts only consume the committed PNGs.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

17 KESTREL-RV32I Hardware Architecture Specification Index

Version: 0.1 **Date:** 2026-10-07 **Purpose:** High-level hardware architecture specification for the kestrel-rv32i single-cycle RV32I core

17.1 Document Organization

Note: All chapters linked below for automated document generation.

17.1.1 Front Matter

- [Document Information](#)

17.1.2 Chapter 1: Introduction

- [Purpose and Scope](#)

17.1.3 Chapter 2: System Overview

- [System Overview and Context](#)

17.1.4 Chapter 3: Architecture

- [ISA Scope](#)
- [Programmer-Visible Machine State](#)

17.1.5 Chapter 4: Interfaces

- [kestrel_core Port Specification](#)
- [Memory Contract](#)
- [Halt and Trap Behavior](#)
- [kestrel_mem_loader and AXIL Programming Interface](#)

17.1.6 Chapter 5: Performance

- [Performance Characteristics](#)

17.1.7 Chapter 6: Integration

- [Integration Guide](#)
-

17.2 Related Documentation

- [KESTREL MAS](#) - Micro-Architecture Specification (block-level implementation)
 - [Simplified RV32I: kestrel](#) - Tutorial-study book this core is documented by
 - [RISC-V Falcon Suite README](#) - Suite overview and rung ladder
 - [RISC-V Unprivileged ISA](#) - ISA authority (cited by chapter)
 - [Testplans](#) - V&V cross-reference (six plans)
-

Last Updated: 2026-10-07 **Maintained By:** RTL Design Sherpa Project

18 Simplified RV32I: kestrel

Source: The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (riscv-spec.pdf, chapter-cited; this book cites chapters and sections, not pages) and the falcon-suite design documentation **Edition:** Simplified for the kestrel-rv32i single-cycle core, rung 1 of the falcon suite. Derived from condensed study notes; emphasis is on what the as-built kestrel core actually does and verifies, with the RISC-V specification cited as the authority. **Version:** 0.1 **Last Updated:** 2026-10-06 **Status:** Draft. Personal study notes.

18.1 Overview

RV32I is the RISC-V base integer instruction set: thirty-seven instructions, six instruction formats, thirty-one general-purpose registers plus the hard-wired zero register, and a program counter. It is small enough to hold in your head and rich enough to run real compiled code. That combination makes it the canonical teaching ISA — and the foundation rung of the falcon suite.

kestrel is the suite's rung-1 core: a single-cycle RV32I implementation in which one instruction retires per clock and the whole machine — fetch, decode, register read, execute, memory access, writeback — completes inside that clock. There is no pipeline, no hazard logic, no trap machinery, and almost no control state. What there is, is a datapath you can trace by hand and a decode truth table you can read top to bottom.

This book condenses RV32I into the material needed to understand kestrel: the instruction set itself (Chapter 2), the datapath (Chapter 3), the control truth table (Chapter 4), the memory contract including the hardware-misaligned policy (Chapter 5), the verification story — the rv32ui battery, spike lockstep, and the riscv-formal proof that caught a bug forty-two directed tests missed (Chapter 6) — and what the single-cycle discipline costs, which is exactly what motivates rung 2, merlin (Chapter 7).

18.2 Document Structure

18.2.1 Chapter 1: Introduction

- [01_introduction.md](#) - What this book is, what kestrel is, how to read the as-built core
- [02_the_falcon_ladder.md](#) - The four-rung suite, why kestrel is rung 1, what each rung teaches
- [03_conventions.md](#) - Notation, register names, halt-versus-trap terminology, citations

18.2.2 Chapter 2: The RV32I ISA Quick Reference

- [01_formats.md](#) - The six instruction formats and how immediates are assembled
- [02_the_instruction_set.md](#) - The thirty-seven base instructions, group by group
- [03_system_layer.md](#) - What kestral does with FENCE, ECALL/EBREAK, MRET, and the CSR class

18.2.3 Chapter 3: The kestral Datapath

- [01_overview.md](#) - Block-level tour, the module list, the complete state inventory
- [02_a_cycle_in_detail.md](#) - One clock, end to end: fetch, decode, execute, writeback
- [03_control_state.md](#) - The five state elements and why there is no FSM
- [04_rvfi.md](#) - The RVFI retire interface and how kestral aggregates it

18.2.4 Chapter 4: The Control Truth Table

- [01_the_control_bundle.md](#) - The thirteen decode outputs and the default bundle
- [02_the_decode_table.md](#) - The full decode case as a table, every row

18.2.5 Chapter 5: The Memory Contract

- [01_the_contract.md](#) - The Harvard interface, word-wide buses, combinational reads
- [02_misaligned_data.md](#) - The hardware-misaligned decision: rotation, the retry cycle, RVFI encoding
- [03_ialign_and_halt.md](#) - IALIGN=32, the instruction-side misaligned halt, the four halt causes

18.2.6 Chapter 6: Verification

- [01_testbench_and_golden.md](#) - The cocotb testbench, sampling contract, golden interpreter lockstep
- [02_rv32ui_battery.md](#) - The 42-test rv32ui-p-* battery, results table, verdict protocol
- [03_spike_lockstep.md](#) - Spike 1.1.0 lockstep, the misaligned-build note, stream-diff details
- [04_riscv_formal.md](#) - The riscv-formal flow: wrapper, checks, depths, 42/42

- [05_the_cex_story.md](#) - The misaligned-jump counterexample: a real bug the battery missed

18.2.7 Chapter 7: What kestrel Costs

- [01_what_kestrel_costs.md](#) - CPI 1 and its price: the whole machine on the critical path
 - [02_merlin_teaser.md](#) - The five-stage answer and what carries over unchanged
-

18.2.8 Version History

Version history

Version	Date	Change
0.1	2026-10-06	First draft

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 kestrel-rv32i Testplans

YAML testplans mapping every kestrel FUB and the core top to the cocotb/pytest tests that exercise them. Format mirrors the pumice/retro_legacy_blocks testplan convention so the same coverage rollup applies (bin/cov_utils/functional_coverage_tracker.py).

19.1 Inventory

Testplan	Module	Test file(s)	Scenarios	Verified	Not impl.
kestrel_regfile_testplan.yaml	kestrel_regfile.v	test_kestrel_regfile.py	5	5	0
kestrel_alu_testplan.yaml	kestrel_alu.sv	test_kestrel_decode.py	5	5	0
kestrel_imm_gen_testplan.yaml	kestrel_imm_gen.sv	test_kestrel_decode.py	6	6	0
kestrel_decode_testplan.yaml	kestrel_decode.sv	test_kestrel_decode.py	12	12	0
kestrel_mem_loader_testplan.yaml	kestrel_mem_loader.sv	test_kestrel_mem_loader.py	9	9	0
kestrel_core_testplan.yaml	kestrel_core.sv	test_kestrel_{core,branch,ls,r,rv32ui,mem_loader,fuzz}.py	18	17	1
Total			55	54	1

Not-implemented scenarios and why:

- **CORE-18** — functional coverage closure / cover model (**Phase C**).

19.2 What each plan covers

- **RF (kestrel_regfile)**: reset-to-zero, write/readback on both ports, read-during-write old-value, x0 discard, independent port addressing (attributed to the core-level golden trace, RF-05).

- **ALU (kestrel_alu):** exhaustive 10-op x 16-value x 16-value golden-model sweep including shamt=b[4:0], signed/unsigned compare flags. The flags are unconnected inside kestrel_core by design (branch comparator is separate) — verified at unit level only.
- **IG (kestrel_imm_gen):** I/S/B/U/J immediate assembly and sign extension with directed $\pm$ vectors; B/J LSB-zero rule; plus a SEED-driven randomized sweep (625 vectors, opcode+funct3 anchored per format class) against the Python imm_golden.
- **DEC (kestrel_decode):** the 48-encoding golden truth table (37 canonical RV32I + mret + fence/fence.i + ecall/ebreak + all 6 CSR-stub forms), illegal / reserved encodings landing on halt cause 0xF, ecall/ebreak causes 1/2. Halt cause 3 (IALIGN) is decode-invisible by design and lives in the CORE plan (CORE-13).
- **ML (kestrel_mem_loader):** AXIL window over both 64 KB regions, per-byte wstrb merge, CTRL.run readback/release sequencing, load-mode isolation (core_rst_n hold, parked PC, no retirement), core  dmem cross-mux both directions, run-mode observability through the AXIL port, AXIL backpressure (bready/rready held low — bvalid/rvalid hold, busy taps, no lost beat), run-mode loader-write rejection (post-CTRL.run poison write leaves the arrays untouched, B still OKAY), and the golden-trace board path (battery images streamed over AXIL with spike lockstep).
- **CORE (kestrel_core):** ISA classes via golden trace, branches/jumps (incl. IALIGN halt cause 3), loads/stores incl. the cross-word 2-cycle retry, the system layer (FENCE NOPs, ECALL/EBREAK halt+hold, illegal halt, trap-beat shape), RVFI invariants, the 42-image rv32ui battery, spike lockstep, and the CORE-17 constrained-random fuzz (seeded streams vs the interpreter full-field diff plus spike lockstep at every level). Phase C gap (coverage closure) is tracked as not_implemented.

19.3 JUnit naming convention (observed)

Kestrel's pytest wrappers launch one Verilator build per (cocotb testcase, test level) cell; each cell writes a cocotb JUnit XML whose testcase name is the **cocotb test function name**, with the pytest module as classname:

```
<testcase name="cocotb_test_kestrel_core_focus"
classname="test_kestrel_core" .../>
```

So test_function strings in these plans quote the cocotb_test_* node names exactly (e.g. cocotb_test_kestrel_branch_all, cocotb_test_kestrel_rv32ui_battery). They

match the tracker by exact name — there are no pytest-parametrized [case-level] node names in the cocotb XML.

Results XML layout: the wrappers set COCOTB_RESULTS_FILE to dv/tests/logs/results_<testcase>_<level>.xml. Both dv/tests/logs/ and dv/tests/local_sim_build/ are gitignored — nothing is checked in; every run regenerates them. The per-cell cocotb XMLs also land one level down at dv/tests/local_sim_build/<testcase>_<level>/<rand>_results.xml with identical testcase names, so to feed the tracker from a previous local run, flatten its XMLs first:

```
cd projects/components/riscv-ip/kestrel-rv32i/dv/tests
find local_sim_build -name '*_results.xml' -exec cp {} logs/ \;
```

19.4 Rollup

```
# From the repo root, after a regression (or after flattening a
previous
# local run's XMLs into dv/tests/logs/ as above):
python3 bin/cov_utils/functional_coverage_tracker.py \
    --testplans-dir
projects/components/riscv-ip/kestrel-rv32i/dv/testplans \
    --report \
    --results-dir
projects/components/riscv-ip/kestrel-rv32i/dv/tests/logs
```

The tracker exists for per-scenario status verification: it matches each YAML's test_function strings against the JUnit XML testcase names and reports whether the referenced cocotb tests actually ran and passed. Its functional-coverage percentage shows 0% for these plans — by design, as in the pumice house: these YAMLs carry no coverage_points blocks, so the tool has no cover data to score. The scenario-level rollup for kestrel is each YAML's implied_coverage block: scenario_tracked counts status: verified scenarios and implied_percentage = tracked / total_scenarios. Per-scenario covers_lines attribution from Verilator .dat files is the pumice update_testplan_coverage.py workflow and is deliberately not wired in kestrel yet (CORE-18 owns the cover model).

19.5 Running the tests

```
cd projects/components/riscv-ip/kestrel-rv32i/dv/tests
make list # discovered test roots
make run-all-func-parallel # FUNC regression (gate+func cells)
make run-all-full-parallel # FULL regression (gate+func+full cells)
```

REG_LEVEL semantics: GATE runs gate cells only, FUNC adds func, FULL adds full. The rv32ui battery reads TEST_LEVEL inside the sim (gate = 3-image smoke, func = 42 images + spike lockstep, full = 42 images without lockstep).

19.6 Functional coverage (CORE-18)

Collection is wired through the central `cov_utils` `conftest` hook (`dv/tests/conftest.py`) — the same pattern every other component uses. To regenerate:

```
make clean-all
COVERAGE=1 make run-all-full
make coverage-report
```

Measured 2026-10-07 across all 60 cells (including the CORE-17 fuzz at full level, 200 streams):

Metric	Result	House target	Status
Line (Verilator)	94.9%	80%	PASS
Line — kestrel_core	100.0%	—	—
Line — kestrel_decode	95.0%	—	—
Line — kestrel_mem_loader	90.1%	—	—

Toggle collection is enabled (`--coverage-toggle`); the house `merge_testlevel_coverage.py` report tracks line only — toggle numbers are ad-hoc from `coverage_data/merged/latest_merged_coverage.dat` via `verilator_coverage`. Known toggle laggards are data-path width toggles exercised thinly by directed tests; the fuzz's random patterns are the planned ratchet. Coverage artifacts (`coverage_data/`, `coverage_reports/`, `local_sim_build/`, `logs/`) are gitignored by design; the numbers above are the committed record, regenerable with the commands in this section.